1. Purpose of This Document

This is a build specification for **KhetConnect**, the hyperlocal farm-labour marketplace app. It is written so it can be pasted into GitHub Copilot Chat (or Copilot Workspace) section-by-section to generate production-quality code. It covers, in exact detail:

1. Authentication error messages (wrong password, unregistered number, duplicate registration)
2. Full API authentication/authorization (no endpoint accessible without a valid JWT, with a proper "Please login first" redirect page)
3. Manual location capture ("Get My Location" button, not auto-detect) with a success tick mark
4. 5 km radius matching engine (farmers ⇄ labourers)
5. Posting flow for both roles, with first-time full form and returning-user pre-filled templates
6. Push notifications via Firebase Cloud Messaging (FCM) triggered on "Accept"
7. A global error-handling standard so **no screen ever breaks, hangs, or shows a blank/white page**

Each section has: the exact user-facing message, the backend exception/response contract, and a Copilot-ready prompt or code skeleton.

---

2. High-Level Architecture

```
React (Vite/CRA) SPA
   │  Axios (JWT in Authorization header, refresh via interceptor)
   ▼
Spring Boot 3 REST API
   │  Spring Security (JWT filter, RSA-signed access tokens)
   │  Global @ControllerAdvice exception handler → uniform JSON error contract
   ▼
MySQL (Users, Posts, Locations, FCM Tokens, Notifications)
   │
Firebase Cloud Messaging (push notifications)
```

Golden rule for Copilot: every controller method must assume the request can fail in a *specific, named* way, and every failure must map to one human-readable message defined in Section 8's catalog — never a generic "Something went wrong."

---

3. Authentication Module

3.1 Registration — Duplicate Phone Number

Trigger: user submits registration form with a phone number that already exists in the `users` table.

Required user-facing message: > "Phone number already exists. Please use another number or login instead."

Backend contract:

```
// UserService.java
public User register(RegisterRequest req) {
    if (userRepository.existsByPhoneNumber(req.getPhoneNumber())) {
        throw new DuplicatePhoneException(
            "Phone number already exists. Please use another number or login instead.");
    }
    // proceed: hash password, save user, issue tokens
}
```

```
// DuplicatePhoneException.java
@ResponseStatus(HttpStatus.CONFLICT) // 409
public class DuplicatePhoneException extends RuntimeException {
    public DuplicatePhoneException(String message) { super(message); }
}
```

React handling: on `409`, show the message inline under the phone field (red text) **and** show a “Login instead” link right next to it — don’t make the user re-type anything.

3.2 Login — Wrong Password vs Not Registered

These must be **two different messages**, not a single vague “Invalid credentials,” because the user needs to know whether to retype the password or go register.

Scenario	HTTP Status	Message
Phone number not found in DB	`404`	“This number is not registered. Please register first.”
Phone number found, password mismatch	`401`	“Phone number or password is incorrect. Please try again.”
Account exists but locked/disabled	`403`	“Your account is temporarily locked. Contact support.”

> Note: For the password-mismatch case specifically, keep the wording as “phone number or password is incorrect” (not “password is wrong”) — this is a standard security practice so an attacker can’t use your login form to enumerate which phone numbers are registered. The *not-registered* case is safe to be explicit about here since your users need that guidance and this app isn’t a public login-guessing target the way a bank is — but you can tighten this later if needed.

```
// AuthService.java
public LoginResponse login(LoginRequest req) {
    User user = userRepository.findByPhoneNumber(req.getPhoneNumber())
        .orElseThrow(() -> new UserNotRegisteredException(
            "This number is not registered. Please register first."));

    if (!passwordEncoder.matches(req.getPassword(), user.getPasswordHash())) {
        throw new InvalidCredentialsException(
            "Phone number or password is incorrect. Please try again.");
    }

    if (!user.isEnabled()) {
        throw new AccountLockedException("Your account is temporarily locked. Contact support.");
    }

    return tokenService.issueTokens(user);
}
```

```
@ResponseStatus(HttpStatus.NOT_FOUND)
public class UserNotRegisteredException extends RuntimeException {
    public UserNotRegisteredException(String m) { super(m); }
}

@ResponseStatus(HttpStatus.UNAUTHORIZED)
public class InvalidCredentialsException extends RuntimeException {
    public InvalidCredentialsException(String m) { super(m); }
}
```

React handling: - `404` → show message + a “Register now” button that pre-fills the phone number on the registration screen. - `401` → show message under the password field, clear the password field, keep focus there.

3.3 Protecting Every API — “Please Login First”

Requirement: no page or API should ever render broken/blank content when the user isn’t authenticated. Every protected

route must redirect to a dedicated screen.

Backend — Spring Security config:

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
          .csrf(csrf -> csrf.disable())
          .authorizeHttpRequests(auth -> auth
              .requestMatchers("/api/auth/**").permitAll()
              .requestMatchers("/api/public/**").permitAll()
              .anyRequest().authenticated()
          )
          .exceptionHandling(ex -> ex
              .authenticationEntryPoint((request, response, authException) -> {
                  response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
                  response.setContentType("application/json");
                  response.getWriter().write("""
                      {"error":"UNAUTHENTICATED","message":"Please login first to continue."}
                      """);
              })
          )
          .sessionManagement(sm -> sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
          .addFilterBefore(jwtAuthFilter(), UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }
}
```

Frontend — Axios interceptor + route guard (this is the part that actually prevents “broken page” symptoms):

```
// api/axiosClient.js
axiosClient.interceptors.response.use(
  (res) => res,
  (err) => {
    if (err.response?.status === 401) {
      // token missing/expired/invalid — never let the page render half-broken
      localStorage.removeItem("accessToken");
      window.location.href = "/login?reason=session_expired";
    }
    return Promise.reject(err);
  }
);
```

```
// components/PrivateRoute.jsx
function PrivateRoute({ children }) {
  const token = localStorage.getItem("accessToken");
  if (!token) {
    return <Navigate to="/login?reason=please_login" replace />;
  }
  return children;
}
```

```
// pages/LoginPage.jsx (top of component)
const [searchParams] = useSearchParams();
const reason = searchParams.get("reason");
const banner =
  reason === "please_login" ? "Please login first to continue." :
  reason === "session_expired" ? "Your session expired. Please login again." :
  null;
```

This guarantees: any attempt to view a protected page without a valid token **always** lands on `/login` with a clear banner — never a blank screen, never a broken API call visible to the user.

---

4. Location Capture — Manual “Get My Location” Button

Requirement change: remove auto-detect-on-load behaviour. Location must be fetched **only** when the user taps “Get My Location,” and a tick mark (✓) must confirm success. No error should leave the user stuck.

4.1 UI State Machine

State	Button Label	Icon
Idle	“Get My Location”	📍 (neutral)
Fetching	“Detecting location…”	spinner
Success	“Location captured”	✅ green tick
Denied	“Location access denied — tap to retry”	⚠ retry
Unavailable/timeout	“Couldn’t get location — tap to retry”	⚠ retry

4.2 React Implementation

```
// hooks/useGetLocation.js
import { useState, useCallback } from "react";

export function useGetLocation() {
  const [status, setStatus] = useState("idle"); // idle | fetching | success | error
  const [coords, setCoords] = useState(null);
  const [errorMsg, setErrorMsg] = useState(null);

  const getLocation = useCallback(() => {
    if (!navigator.geolocation) {
      setStatus("error");
      setErrorMsg("Your browser doesn't support location. Please enter manually or try another browser.");
      return;
    }

    setStatus("fetching");
    navigator.geolocation.getCurrentPosition(
      (pos) => {
        setCoords({ lat: pos.coords.latitude, lng: pos.coords.longitude });
        setStatus("success");
      },
      (err) => {
        setStatus("error");
        if (err.code === err.PERMISSION_DENIED) {
          setErrorMsg("Location access denied. Please allow location permission in your browser settings and try again."
        } else if (err.code === err.TIMEOUT) {
          setErrorMsg("Location request timed out. Please check your GPS/network and try again.");
        } else {
          setErrorMsg("Couldn't get your location. Please try again.");
        }
      },
      { enableHighAccuracy: true, timeout: 10000, maximumAge: 0 }
    );
  }, []);

  return { status, coords, errorMsg, getLocation };
}
```

```
// components/GetLocationButton.jsx
function GetLocationButton({ onLocationReady }) {
  const { status, coords, errorMsg, getLocation } = useGetLocation();

  useEffect(() => {
    if (status === "success" && coords) onLocationReady(coords);
  }, [status, coords]);

  return (
    <div className="location-capture">
      <button type="button" onClick={getLocation} disabled={status === "fetching"}>
        {status === "fetching" && "Detecting location…"}
        {status === "success" && "✅ Location captured"}
        {(status === "idle") && "📍 Get My Location"}
        {status === "error" && "⚠️ Tap to retry"}
      </button>
      {status === "error" && <p className="error-text">{errorMsg}</p>}
    </div>
  );
}
```

Post-submit rule: the “Post” button for both farmer and labour forms must be **disabled until** `status === "success"`, so a post can never be saved without coordinates.

---

5. Geo-Matching Engine — 5 km Radius

Requirement: labourers see only farmer posts within 5 km of their own captured location, and farmers see only labourer posts within 5 km — symmetric in both directions.

5.1 Database — store coordinates as columns (simplest, fastest for a fresher-level monolith)

```
ALTER TABLE posts ADD COLUMN latitude DOUBLE NOT NULL;
ALTER TABLE posts ADD COLUMN longitude DOUBLE NOT NULL;
ALTER TABLE users ADD COLUMN latitude DOUBLE;
ALTER TABLE users ADD COLUMN longitude DOUBLE;
CREATE INDEX idx_posts_lat_lng ON posts(latitude, longitude);
```

5.2 Haversine Query (native SQL, MySQL)

```
// PostRepository.java
@Query(value = """
    SELECT *, (
      6371 * acos(
        cos(radians(:lat)) * cos(radians(latitude)) *
        cos(radians(longitude) - radians(:lng)) +
        sin(radians(:lat)) * sin(radians(latitude))
      )
    ) AS distance_km
    FROM posts
    WHERE post_type = :targetType
      AND status = 'ACTIVE'
    HAVING distance_km <= 5
    ORDER BY distance_km ASC
    """, nativeQuery = true)
List<Post> findNearbyPosts(@Param("lat") double lat,
                           @Param("lng") double lng,
                           @Param("targetType") String targetType);
```

`targetType` is `"LABOUR_NEED"` when a labourer is browsing (they see farmer posts) or `"FARMER_AVAILABLE"` — adapt the two enum values to your actual `PostType`. The key symmetric rule for Copilot:

- If `currentUser.role == FARMER` → query posts where `postType == LABOUR_OFFER` (labour availability posts)
- If `currentUser.role == LABOURER` → query posts where `postType == WORK_NEED` (farmer job posts)

- Both queries use the **same 5 km Haversine filter** against the current user's last captured coordinates.

5.3 Fallback message when nothing is nearby

> "No posts within 5 km right now. We'll notify you as soon as someone nearby posts."

Never show a blank list with no explanation — always render this empty-state message.

---

6. Posting Flow — First-Time Form vs Saved Template

6.1 Rule

- **First post ever** (no `PostTemplate` row exists for this user) → show the **full form**: name, phone (pre-filled from profile), village/area, workers count / wage / date (farmer) or skills / daily rate / availability (labourer), location capture.
- **Every post after that** → show the **template screen**: last-used values pre-filled, with an "Edit" pencil icon on each field and a single "Post" button. Location is always re-captured fresh (never reused from the old template) since the user's location may have changed.

6.2 Data model

```
@Entity
public class PostTemplate {
    @Id @GeneratedValue
    private Long id;

    @OneToOne
    private User user;

    private Integer workersCount;
    private Double wagePerDay;
    private String preferredDate;
    private String notes;

    private LocalDateTime lastUsedAt;
}
```

```
// PostService.java
public PostFormResponse getPostForm(User user) {
    return postTemplateRepository.findByUser(user)
        .map(t -> new PostFormResponse(true, t)) // isReturning = true, pre-filled
        .orElse(new PostFormResponse(false, null)); // isReturning = false, blank form
}

public Post createPost(User user, PostRequest req) {
    validatePostRequest(req); // see 6.3
    Post post = postRepository.save(Post.from(user, req));
    postTemplateRepository.save(PostTemplate.from(user, req)); // upsert last-used values
    return post;
}
```

6.3 Field Validation & Exact Error Messages

Field	Failure condition	Message
Workers count / labour count	Empty or 0	"Please enter how many workers are needed."
Wage / daily rate	Empty, 0, or negative	"Please enter a valid daily wage."
Date	Empty or in the past	"Please select today or a future date."
Location	Not captured (`status != success`)	"Please tap 'Get My Location' before

Location	Not captured (`status !== success`)	"Please tap 'Get My Location' before posting."
Phone (if editable)	Invalid format	"Please enter a valid 10-digit phone number."
Any post submit	Network/server error	"Couldn't post right now. Please check your connection and try again."
Any post submit	Server 500	"Something went wrong on our end. Your post wasn't saved — please try again."

```
private void validatePostRequest(PostRequest req) {
    if (req.getWorkersCount() == null || req.getWorkersCount() <= 0)
        throw new InvalidPostException("Please enter how many workers are needed.");
    if (req.getWagePerDay() == null || req.getWagePerDay() <= 0)
        throw new InvalidPostException("Please enter a valid daily wage.");
    if (req.getPreferredDate() == null || req.getPreferredDate().isBefore(LocalDate.now()))
        throw new InvalidPostException("Please select today or a future date.");
    if (req.getLatitude() == null || req.getLongitude() == null)
        throw new InvalidPostException("Please tap 'Get My Location' before posting.");
}
```

---

7. Push Notifications — FCM on "Accept"

Requirement: when a farmer accepts a labourer's offer (or vice versa), both sides get a push notification showing the other person's details (name, phone, distance).

7.1 Store FCM tokens

```
@Entity
public class FcmToken {
    @Id @GeneratedValue
    private Long id;

    @ManyToOne
    private User user;

    private String token;
    private LocalDateTime registeredAt;
}
```

Frontend registers the token once on login/app load:

```
// firebase-messaging.js
import { getMessaging, getToken } from "firebase/messaging";

export async function registerFcmToken() {
  const messaging = getMessaging();
  const permission = await Notification.requestPermission();
  if (permission !== "granted") return; // don't block the app if denied

  const token = await getToken(messaging, { vapidKey: import.meta.env.VITE_FCM_VAPID_KEY });
  await axiosClient.post("/api/fcm/register", { token });
}
```

7.2 Trigger on Accept (backend)

```
// AcceptService.java
@Transactional
public void acceptOffer(Long postId, User acceptingUser) {
    Post post = postRepository.findById(postId)
        .orElseThrow(() -> new PostNotFoundException("This post is no longer available."));

    if (post.getStatus() != PostStatus.ACTIVE)
        throw new PostAlreadyTakenException("This post has already been accepted by someone else.");

    post.setStatus(PostStatus.ACCEPTED);
    post.setAcceptedBy(acceptingUser);
    postRepository.save(post);

    User postOwner = post.getUser();
    notificationService.sendAcceptNotification(postOwner, acceptingUser, post);
    notificationService.sendConfirmationNotification(acceptingUser, postOwner, post);
}
```

```
// NotificationService.java
public void sendAcceptNotification(User recipient, User counterpart, Post post) {
    String fcmToken = fcmTokenRepository.findTopByUserOrderByRegisteredAtDesc(recipient)
        .map(FcmToken::getToken).orElse(null);
    if (fcmToken == null) return; // fail silently, don't break the accept flow

    Message message = Message.builder()
        .setToken(fcmToken)
        .setNotification(Notification.builder()
            .setTitle("Your post was accepted!")
            .setBody(counterpart.getName() + " (" + counterpart.getPhoneNumber() + ") accepted your post.")
            .build())
        .putData("counterpartName", counterpart.getName())
        .putData("counterpartPhone", counterpart.getPhoneNumber())
        .putData("postId", post.getId().toString())
        .build();

    try {
        FirebaseMessaging.getInstance().send(message);
    } catch (FirebaseMessagingException e) {
        log.warn("FCM send failed for user {}: {}", recipient.getId(), e.getMessage());
        // never throw — notification failure must not break the accept transaction
    }
}
```

Key resilience rule for Copilot: notification sending must be wrapped so a failed push **never rolls back or breaks** the accept transaction — the accept itself is the source of truth; the notification is best-effort.

7.3 Details Shown on Accept (in-app, not just push)

When either side accepts, show an in-app confirmation card immediately (don’t wait for the push): - Name, phone number (tap-to-call), distance in km, and post details of the other party.

---

8. Global Error-Handling Standard

8.1 Uniform API Error Contract

Every error response — no exceptions — follows this shape so the frontend can handle it generically:

```
{
  "error": "DUPLICATE_PHONE",
  "message": "Phone number already exists. Please use another number or login instead.",
  "status": 409,
  "timestamp": "2026-08-05T10:15:30"
}
```

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(DuplicatePhoneException.class)
    public ResponseEntity<ErrorResponse> handle(DuplicatePhoneException e) {
        return build(HttpStatus.CONFLICT, "DUPLICATE_PHONE", e.getMessage());
    }

    @ExceptionHandler(UserNotRegisteredException.class)
    public ResponseEntity<ErrorResponse> handle(UserNotRegisteredException e) {
        return build(HttpStatus.NOT_FOUND, "USER_NOT_REGISTERED", e.getMessage());
    }

    @ExceptionHandler(InvalidCredentialsException.class)
    public ResponseEntity<ErrorResponse> handle(InvalidCredentialsException e) {
        return build(HttpStatus.UNAUTHORIZED, "INVALID_CREDENTIALS", e.getMessage());
    }

    @ExceptionHandler(InvalidPostException.class)
    public ResponseEntity<ErrorResponse> handle(InvalidPostException e) {
        return build(HttpStatus.BAD_REQUEST, "INVALID_POST", e.getMessage());
    }

    @ExceptionHandler(PostAlreadyTakenException.class)
    public ResponseEntity<ErrorResponse> handle(PostAlreadyTakenException e) {
        return build(HttpStatus.CONFLICT, "POST_ALREADY_TAKEN", e.getMessage());
    }

    @ExceptionHandler(Exception.class) // catch-all, last line of defence
    public ResponseEntity<ErrorResponse> handleUnknown(Exception e) {
        log.error("Unhandled exception", e);
        return build(HttpStatus.INTERNAL_SERVER_ERROR, "SERVER_ERROR",
            "Something went wrong on our end. Please try again in a moment.");
    }

    private ResponseEntity<ErrorResponse> build(HttpStatus status, String code, String msg) {
        return ResponseEntity.status(status)
            .body(new ErrorResponse(code, msg, status.value(), LocalDateTime.now()));
    }
}
```

This catch-all is the single most important line in the whole spec — it guarantees the API can never leak a raw stack trace or crash a page; the frontend always gets a friendly message no matter what breaks internally.

8.2 Frontend Global Safety Net

```
// components/ErrorBoundary.jsx
class ErrorBoundary extends React.Component {
  state = { hasError: false };
  static getDerivedStateFromError() { return { hasError: true }; }
  componentDidCatch(error, info) { console.error(error, info); }
  render() {
    if (this.state.hasError) {
      return (
        <div className="fullscreen-error">
          <p>Something went wrong. Please refresh the page.</p>
          <button onClick={() => window.location.reload()}>Refresh</button>
        </div>
      );
    }
    return this.props.children;
  }
}
```

Wrap the root `<App />` in this. Combined with the Axios interceptor (Section 3.3) and the uniform error contract (8.1), this closes all three failure surfaces: **network errors, auth errors, and unexpected JS crashes** — the app should never show a blank white screen.

8.3 Full Error Message Catalog (for Copilot to reference verbatim)

Code	Status	Message

`DUPLICATE_PHONE`	409	Phone number already exists. Please use another number or login instead.
`USER_NOT_REGISTERED`	404	This number is not registered. Please register first.
`INVALID_CREDENTIALS`	401	Phone number or password is incorrect. Please try again.
`ACCOUNT_LOCKED`	403	Your account is temporarily locked. Contact support.
`UNAUTHENTICATED`	401	Please login first to continue.
`INVALID_POST`	400	(field-specific message from Section 6.3)
`POST_ALREADY_TAKEN`	409	This post has already been accepted by someone else.
`POST_NOT_FOUND`	404	This post is no longer available.
`LOCATION_DENIED`	— (client-side)	Location access denied. Please allow location permission and try again.
`NO_NEARBY_POSTS`	— (client-side empty state)	No posts within 5 km right now. We'll notify you as soon as someone nearby posts.
`SERVER_ERROR`	500	Something went wrong on our end. Please try again in a moment.
`NETWORK_ERROR`	— (client-side)	Couldn't connect. Please check your internet and try again.

---

9. Copilot Prompting Strategy

Feed this document into Copilot in this order for best results:

1. Paste Section 3 first → generate `AuthService`, `SecurityConfig`, exceptions, and `GlobalExceptionHandler` skeleton.
2. Paste Section 8.1 next → "extend the GlobalExceptionHandler with these additional handlers" so all exceptions funnel into one consistent file.
3. Paste Section 4 → generate `useGetLocation` hook and `GetLocationButton` component exactly as specified (don't let Copilot auto-run geolocation on mount).
4. Paste Section 5 → generate the repository query and matching service; confirm the symmetric farmer/labourer logic explicitly in your prompt.
5. Paste Section 6 → generate `PostTemplate` entity, form-branching logic, and validation.
6. Paste Section 7 last → FCM registration + accept-flow notification service, emphasizing the "never break the transaction" rule.

One instruction to give Copilot every time: *"Every exception must extend RuntimeException and be annotated or handled centrally — never let a raw exception reach the client. Every network call on the frontend must have a loading, success, and error UI state — never leave a screen in an undefined state."*

---

10. Acceptance Checklist (test before calling it done)

- ☐ Register with an existing phone number → correct duplicate message, no crash
- ☐ Login with unregistered number → "please register first" message + working register link
- ☐ Login with wrong password → correct message, password field clears
- ☐ Open any protected page/URL directly while logged out → redirected to `/login` with "Please login first" banner, never a blank page
- ☐ Token expires mid-session → auto-redirect to login with "session expired" banner
- ☐ Tap "Get My Location" → spinner → green tick on success; deny permission → clear retry message

- ☐ “Post” button stays disabled until location tick appears
- ☐ Farmer posting sees only labourer posts within 5 km, and vice versa — verify with two test accounts at known coordinates
- ☐ No nearby posts → friendly empty-state message, not a blank list
- ☐ First-ever post → full form; second post onward → pre-filled template with edit option
- ☐ Submitting invalid post fields → exact per-field message from Section 6.3
- ☐ Accept a post → both users get FCM push + in-app card with counterpart’s name, phone, distance
- ☐ Kill network mid-request → “Couldn’t connect” message, not a frozen spinner
- ☐ Force a 500 on the backend → generic friendly message, no stack trace visible, app doesn’t crash